

AI-Powered Autonomous Restaurant System

- Multi-Robot Fleet Management
- Computer Vision Food Inspection
- Voice-Activated Ordering

ROS2 Humble | Nav2 | YOLOv8 | OpenAI APIs

Project Overview

Team Composition

- 6 Members: SW Developers + HW Integration
- Development Period: 8 weeks

Key Metrics

- 10,225+ Lines of Code
- 155 Test Cases
- 3 Mobile Robots (TurtleBot3)
- 2 Robot Arms (MyCobot 280)

Full Stack: ROS2 + Backend + AI + GUI

System Architecture

Core Components

- FMS (Fleet Management System) - Central Control
- Mobile Robots - pinky1, pinky2, pinky3
- Robot Arms - Sandwich Assembly
- AI Servers - YOLO + Voice Recognition
- GUI - Customer Kiosk + Admin Panel

Master PC (192.168.1.3) → Domain Bridge → Robots

ROS2 Multi-Domain Architecture

Domain ID Assignment

- Domain 25: FMS (Master)
- Domain 11: pinky1
- Domain 12: pinky2
- Domain 13: pinky3
- Domain 20: Robot Arms

Domain Bridge for Cross-Domain Communication

Problem #1: DDS Discovery Failure

Symptom

- ros2 topic list shows nothing from robots
- SSH connection works fine
- ping successful but no ROS2 communication

Discovery Process

- Wireshark: No multicast packets on WiFi
- Router logs: Multicast UDP blocked

WiFi AP blocks multicast by default!

Solution: CycloneDDS Unicast Discovery

Configuration

```
<Peer address='192.168.1.7' /> <!-- pinky1 -->
```

```
<Peer address='192.168.1.6' /> <!-- pinky2 -->
```

Environment Setup

```
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

```
export CYCLONEDDS_URI=file:///path/to/cyclonedds.xml
```

Direct IP-to-IP communication - Works on any WiFi!

Problem #2: ROS2 Topic Name Collision

Issue

- All robots publish /cmd_vel, /odom, /scan
- Single domain = mixed data
- Which /amcl_pose belongs to which robot?

Impact

- Navigation commands sent to wrong robot
- Localization data corrupted
- Fleet management impossible

Same topic names, different robots!

Solution: Domain Bridge Architecture

Approach

- Separate Domain ID per robot
- Domain Bridge remaps topics

Topic Remapping

```
/cmd_vel (Domain 11) → /pinky1/cmd_vel (Domain 25)
```

```
/amcl_pose (Domain 11) → /pinky1/amcl_pose (Domain 25)
```

Clean namespace isolation with centralized control

Navigation: Nav2 + AMCL

Stack Components

- AMCL: Adaptive Monte Carlo Localization
- Nav2: Path Planning + Obstacle Avoidance
- TF2: Coordinate Frame Management

Waypoints

- Kitchen: point13 (food pickup)
- Tables: table1 ~ table4
- Parking: pinky1_spot, pinky2_spot, pinky3_spot

Problem #3: Robot-Cooking Synchronization

Challenge

- Robot arrives at kitchen before food ready
- Food ready but robot still en route
- Race condition in order workflow

Failed Approaches

- Simple timer-based waiting
- Polling status repeatedly

Need: Precise state machine coordination

Solution: Event-Driven Order Workflow

State Transitions

- RECEIVED → COOKING → LOADING → LOADED
- LOADED → DELIVERING → ARRIVED → COMPLETED

AND Condition Logic

```
if robot_at_kitchen AND cooking_complete:
```

```
transition_to(LOADED)
```

Robust synchronization via state machine!

AI Server: YOLOv8 Food Inspection

Capabilities

- Real-time object detection
- Food quality verification
- Assembly completeness check

Integration

```
POST /detect {'image': base64_data}
```

```
Response: {'objects': [...], 'confidence': 0.95}
```

Custom trained model for sandwich ingredients

AI Server: Voice Ordering System

Pipeline

- Whisper API: Speech-to-Text
- GPT-4 Function Calling: Intent Extraction
- TTS: Order Confirmation

Example Flow

```
'■■■■ ■ ■ ■■■' → {menu: 'cheeseburger', qty: 2}
```

Natural language order processing

GUI: Customer & Admin Interfaces

Customer GUI (PyQt5)

- Touch-friendly kiosk interface
- Menu selection with images
- Real-time order tracking
- Delivery notification popup

Admin GUI

- Fleet status monitoring
- Order management
- Robot manual control

Problem #4: GUI-FMS Communication

Challenge

- GUI runs on separate device (tablet)
- ROS2 not available on Android/tablets
- Need bidirectional real-time updates

Requirements

- Push notifications (delivery arrived)
- Reliable message delivery
- Low latency (<100ms)

Solution: Custom TCP/JSON Protocol

Protocol Design

4-byte length header + JSON payload

Message Types

- new_order: Customer places order
- delivery_notification: Push to customer
- delivery_complete: Customer confirms receipt

Thread-per-client model for reliability

Error Detection & Recovery

ErrorDetector Features

- Navigation timeout detection
- Robot stuck detection
- Communication failure handling

Recovery Options

- RETRY: Attempt same action again
- SKIP: Proceed to next step
- MANUAL: Request operator intervention

Graceful degradation for production stability

Testing: Skip Mode Architecture

Problem

- Physical robots not always available
- Need to test FMS logic independently

Skip Mode Features

- Simulated navigation completion
- Mock sensor data
- Accelerated time (configurable delay)

```
ros2 run fms fms_node --ros-args -p skip_mode:=true
```

Database: PostgreSQL Schema

Core Tables

- orders: Order tracking with status
- menu_items: Menu catalog
- robots: Fleet status
- delivery_history: Analytics data

Features

- Timestamp tracking for SLA
- Foreign key relationships
- Index optimization for queries

Performance Characteristics

Latency

- Order acceptance: <100ms
- Push notification: <50ms
- Navigation goal: <200ms

Throughput

- Order processing: ~60 seconds average
- Concurrent robots: 3 (scalable)

Production-ready performance

Key Lessons Learned

Technical Insights

- WiFi multicast is unreliable - use unicast
- Domain separation simplifies multi-robot
- State machines prevent race conditions

Process Insights

- Skip mode enables parallel development
- Good logging saves debugging time
- Integration tests catch real issues

Future Development

Short-term

- Additional robot support (5+ fleet)
- Enhanced voice commands
- Mobile app for customers

Long-term

- Multi-floor navigation
- Autonomous charging
- ML-based demand prediction

Conclusion

Kitchmatics Achievement

- Fully integrated autonomous restaurant system
- Multi-robot coordination with FMS
- AI-powered food inspection & voice ordering
- Production-ready error handling

From concept to working prototype in 8 weeks